

Two Free Android AI Apps on AutoGPT

News Curator + Multilingual Storyteller - end-to-end plan: architecture, free-tier budget, epics, and delivery. Prepared for Ganta Shanak, 14 September 2026.

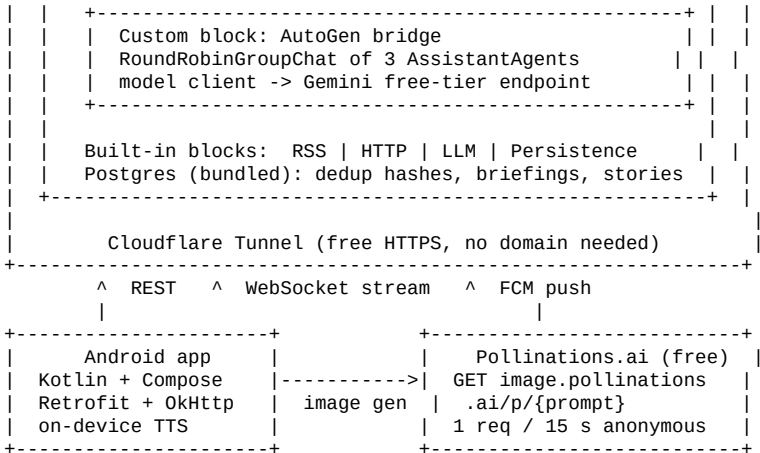
Executive summary

We will build and ship two Android apps at zero recurring cost: a personalized **News & Content Curator** that delivers a daily AI-summarized briefing by push notification, and a **Multilingual Storyteller** that generates illustrated, narrated stories in major Indian languages. Both apps share one cloud backend: a self-hosted AutoGPT Platform on an Oracle Cloud Always Free VM, with Microsoft AutoGen running the multi-agent reasoning inside a custom AutoGPT block, and Gemini's free tier as the LLM. The only paid item in the entire plan is the optional one-time \$25 Google Play listing fee; APK distribution via GitHub is free. Build order: infrastructure first, then the News Curator end-to-end, then the Storyteller, then documentation and delivery.

1. The three repositories and what each contributes

All three are open source and all three carry a real role. AutoGPT is the deployed engine, AutoGen is the reasoning layer inside it, and ECC (Everything Claude Code) is the development methodology that governs how the code is written, tested, reviewed, and delivered.

Repository	Role in the plan	Verified contents we use



Layer	Component	Free provider
Cloud VM	AutoGPT Platform + Postgres + AutoGen, via Docker	Oracle Cloud Always Free (2 OCPU ARM, 12 GB RAM)
Reasoning	3-agent AutoGen team per pipeline	AutoGen + Gemini API free tier

Component	Free source	Limit	Capacity in this plan
Storage	Postgres on the VM	VM disk only (200 GB Always Free block volume)	Years of briefings and stories
Push	Firebase Cloud Messaging	Free, no cap	All users, both apps

Sources: Google AI Gemini rate-limits documentation and pricing guides current to May 2026; Oracle Cloud Always Free resources documentation (limits reduced to 2 OCPU / 12 GB in June 2026); Pollinations.AI API documentation.

Design consequences: dedup by content hash runs before any LLM call so quota is never spent on a repeat; image requests queue with a 15-second spacing; and if the apps are later published to many users, each user brings their own free Gemini key (bring-your-own-key), keeping the operator's cost at zero.

4. Project A - News & Content Curator

A user picks topics and RSS feeds; every morning at 7:00 AM IST an agent team reads the feeds, drops articles already seen, summarizes the rest, and pushes a briefing to the phone.

Agent pipeline

RSS Reader (user feeds) -> HTTP Fetch full text via Jina Reader (free) -> AutoGen bridge: researcher extracts facts, summarizer writes a 2-paragraph summary, editor ranks by relevance -> dedup check in Postgres (SHA-256 of title + body) -> new items stored as a daily briefing -> FCM push notification to the Android app.

Backend additions

- 1 Agent graph assembled once via AutoGPT's builder API; runs are triggered by the built-in scheduler with variable feed lists.
- 2 New REST route `/api/briefings` returns the latest briefing JSON for the app.
- 3 WebSocket stream exposes live run progress for manual refreshes.
- 4 Retry with exponential backoff on Gemini 429 responses.

Android app

- Screens: briefing list, article detail, settings (feeds, topics, server URL).
- Stack: Kotlin, Jetpack Compose, Material 3, Koin DI, Retrofit, OkHttp WebSocket, StateFlow with Turbine tests.
- FCM receiver deep-links a push into the latest briefing.

5. Project B - Multilingual Storyteller

The user types a topic and picks a language (Hindi, Tamil, Telugu, Bengali, Marathi, and others); an agent team writes an illustrated story, and the phone narrates it aloud with on-device TTS at no cost.

Agent pipeline

User prompt (topic + language) from the app -> AutoGen bridge: story_writer drafts in the chosen Indic language, image_prompter turns the mood into an English image prompt, story_editor polishes the text -> HTTP GET to image.pollinations.ai for the cover -> story text and image URL stored in Postgres -> FCM push to the Android app, which narrates the story with on-device TTS.

Key design decision - TTS on device

Cloud TTS free tiers are tiny for Indic languages. Android's built-in TextToSpeech engine, with the locale set to the story's language, is free, unlimited, offline, and private. The agent produces text; the app narrates it. This replaces ElevenLabs entirely.

Android app

- Screens: prompt (topic + language), live generation view with streaming text, story library grid with covers, story reader with play button.
- Stack: same as Project A, plus Coil for cover images.
- TTS manager wraps the platform engine with per-language locales and pause/resume controls.

6. Epic roadmap with checkpoints

Six epics, each ending in a checkpoint commit, following ECC's epic-decompose / feature-dev / code-review cycle. ECC agents listed per epic are the review roles applied during that phase.

Epic 1 - Cloud infrastructure and AutoGPT deployment

ECC review roles: architect, python-reviewer, security-reviewer, build-error-resolver

- 1 Provision the Oracle Always Free ARM VM (Ubuntu) and open ports.
- 2 Install Docker; deploy AutoGPT Platform from docker-compose.platform.yml with bundled Postgres; set the Gemini key as the LLM credential.
- 3 Expose REST + WebSocket over HTTPS via Cloudflare Tunnel.
- 4 Write the AutoGen bridge block (RoundRobinGroupChat of 3 AssistantAgents on the Gemini OpenAI-compatible endpoint) with tests, including 429 retry handling.

Epic 2 - News Curator backend

ECC review roles: planner, tdd-guide, python-reviewer, fastapi-reviewer

- 1 Build the RSS -> fetch -> AutoGen bridge -> dedup -> store -> push graph.
- 2 Schedule the daily 7:00 AM IST run.
- 3 Add /api/briefings and the WebSocket progress stream.

- 4 Tests at 80%+ coverage: RSS parsing, fetch errors, dedup, rate-limit retry.

Epic 3 - News Curator Android app

ECC review roles: kotlin-reviewer, kotlin-build-resolver, security-reviewer, e2e-runner

- 1 Scaffold Kotlin + Compose project with data / domain / ui / di layers.
- 2 Retrofit + WebSocket clients; EncryptedSharedPreferences for the server URL.
- 3 Compose screens and FCM deep-linking.
- 4 Unit, Compose UI, and end-to-end tests for the open -> read -> refresh -> stream flow.

Epic 4 - Storyteller backend

ECC review roles: planner, tdd-guide, python-reviewer, security-reviewer

- 1 Build the prompt -> AutoGen bridge -> Pollinations -> store -> push graph.
- 2 REST /api/stories (create + list) and WebSocket generation stream.
- 3 Image queue with 15-second spacing; free Pollinations registration token stored as a credential.
- 4 Tests for generation, prompt extraction, image retry.

Epic 5 - Storyteller Android app

ECC review roles: kotlin-reviewer, kotlin-build-resolver, security-reviewer, e2e-runner

- 1 Same scaffold as Epic 3 plus Coil image loading.
- 2 Prompt, live-generation, library, and reader screens.
- 3 On-device TTS manager with per-language locales.
- 4 Tests for story creation, stream parsing, TTS setup; E2E for prompt -> generate -> library.

Epic 6 - Documentation and delivery

ECC review roles: doc-updater, code-reviewer, refactor-cleaner

- 1 READMEs: setup, architecture, environment variables, deployment steps.
- 2 Create GitHub repositories (name and visibility confirmed with the user first); commit epic by epic.
- 3 Final code review, dead-code cleanup, security scan.
- 4 Ship APKs via GitHub Releases.

7. Development methodology (from ECC)

- Test-driven: tests precede implementation; 80%+ coverage required per module.
- Security-first: no hardcoded keys, HTTPS-only network config, encrypted storage on Android, secrets in AutoGPT credentials.
- Kotlin rules: val over var, no double-bang null assertions, immutable data classes, Koin injection, JUnit + Turbine + coroutines-test.

- Python rules for backend blocks: typed, validated inputs, FastAPI conventions.
- Every epic ends with code-review and a checkpoint commit.

8. Risks and mitigations

Risk	Impact	Mitigation
Oracle 'out of host capacity' when provisioning	Setup delay (days)	Retry in different availability domains; fallback to running the stack on a home PC behind Cloudflare Tunnel
Gemini free quota exhausted	Briefings or stories fail that day	Hash-dedup before LLM calls; backoff on 429; quotas reset daily at midnight Pacific
Pollinations 15-second image limit	Bursty story generation slows	Server-side queue with spacing; free registration lifts the limit to 1 per 5 s
AutoGen in maintenance mode	No new features upstream	Small, stable API surface used; successor (Microsoft Agent Framework) documented as migration path
Free tiers change over time	Architecture needs adjustment	Every paid-substitute is isolated behind one block or interface, swappable without touching the apps
Indic TTS voice quality varies by device	Uneven narration	